

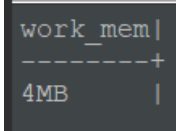
Parte 3:

ia:

1. Seq Scan on producto (hijo, se ejecuta primero) Seq Scan on producto (cost=0.00..1320.02 rows=50002 width=94) (actual time=0.041..5.899 rows=50002.00 loops=1) Buffers: shared hit=820
 - Qué hace: escaneo secuencial (full scan) de la tabla producto, de principio a fin, sin índice.
 - actual time=0.041..5.899: la primera fila se produce a los 0.041 ms y la última a los 5.899 ms. Tarda 5.9 ms en leer las 50.002 filas.
 - rows=50002: devuelve las 50.002 filas (no hay filtro aquí).
 - Buffers: shared hit=820: la tabla entera (~820 bloques $\times$ 8 kB $\approx$ 6,4 MB) ya estaba en memoria (buffer cache compartido). hit = no hubo lectura desde disco. Cero I/O aquí, por eso es tan rápido este nodo.
 - Coste estimado: 0.00..1320.02. El 0.00 es el coste de empezar, 1320.02 el de terminarlo (estimación del planificador, no tiempo real). Conclusión del nodo: es barato y rápido porque cabe en memoria. No hay peligro aquí.
2. WindowAgg (nodo intermedio) WindowAgg (cost=1945.01..1945.05 rows=50002 width=126) (actual time=59.574..100.808 rows=50002.00 loops=1) Window: w1 AS () Storage: Disk Maximum Storage: 5566kB Buffers: shared hit=820, temp read=964 written=697
 - Qué hace: aplica la función de ventana AVG(precio) OVER () y calcula el promedio global de precio para cada fila.
 - Window: w1 AS (): el paréntesis vacío dice que la ventana no tiene PARTITION BY ni ORDER BY. Es UNA sola partición con todas las filas. Eso implica que el nodo es bloqueante: para calcular el promedio global debe esperar a haber visto TODAS las filas antes de poder emitir la primera.
 - Por eso actual time=59.574..100.808: el primer valor de salida no sale hasta los ~59.6 ms (tuvo que consumir el 100% de la entrada para saber el promedio), y termina a los ~100.8 ms.
 - width=126 vs width=94: la fila crece 32 bytes porque añade la columna calculada avg_precio.
 - Storage: Disk / Maximum Storage: 5566kB ← EL PUNTO DÉBIL del plan. El promedio global se ordena/materializa temporalmente, y PostgreSQL necesitaba ~5,4 MB para guardar el estado de la ventana. Como el work_mem es 4 MB (4096 kB), no cupo en memoria y se derramó a disco (spill).
 - temp read=964 written=697: el spill real en bloques de 8 kB: escribió 697 bloques (~5,4 MB) a un archivo temporal y luego leyó 964 (7,5 MB) de vuelta. Ese ir y venir a disco es prácticamente todo el coste de la consulta (los 100 ms). Conclusión del nodo: aquí está el cuello de botella. No es el scan, es la ventana que no cabió en work_mem y pagó disco.
3. Subquery Scan on sub (nodo raíz, se ejecuta al final) Subquery Scan on sub (cost=1945.01..2570.07 rows=16667 width=94) (actual time=59.601..112.129 rows=25094.00 loops=1) Filter: (sub.precio > sub.avg_precio) Rows Removed by Filter: 24908

- Qué hace: envuelve el resultado de la subconsulta y aplica el filtro de la consulta externa: `precio > avg_precio`.
 - `rows=25094`: de las 50.002 filas, 25.094 pasan el filtro (precio por encima del promedio).
 - Rows Removed by Filter: 24908: las otras 24.908 se descartan en este nodo.
 - `actual time=59.601..112.129`: empieza casi al mismo tiempo que el WindowAgg termina de calentar (~59.6 ms) y termina a los ~112 ms. El filtro en sí es trivial (comparación en memoria); lo que paga es recibir las 50.002 filas desde el nodo anterior.
 - Dato interesante — la estimación falló: el planificador estimó `rows=16667` (es decir, ~1/3 de las filas pasando el filtro), pero la realidad fue 25.094 (~50%). Es una estimación de selectividad conservadora sobre la columna `precio` (probablemente sin histograma muy fino). No cambia el plan, pero es una métrica de calidad: las estadísticas no eran precisas.
- Resumen global
- Planning Time: 0.514 ms (despreciable).
 - Execution Time: 113.493 ms.
 - Flujo real: Seq Scan (5.9 ms) → WindowAgg con spill a disco (~59–101 ms, el 85% del tiempo) → Subquery Scan con filtro (59–112 ms). El plan es correcto y razonable para 50.000 filas, pero tiene un desperdicio claro: el spill a disco del WindowAgg.

Las descripciones fueron casi todas correctas, pero asumió datos que no se le habían proporcionado como contexto (aunque en este caso, las acertó):

Storage: Disk / Maximum Storage: 5566kB ← EL PUNTO DÉBIL del plan. El promedio global se ordena/materializa temporalmente, y PostgreSQL necesitaba ~5,4 MB para guardar el estado de la ventana. Como el <code>work_mem</code> es 4 MB (4096 kB), no cupo en memoria y se derramó a disco (spill).	Completo con datos asumidos pero correcta	Nunca se proporcionó contexto sobre el <code>work mem</code> , la ia asumió que ese error se debió a que los supuestos 5,4mb no entraron en los 4mb. Pero en este caso ,si se tenia la memoria por defecto(4mb) 

Parte 4:

1. Redactar una especificación precisa (spec) para al menos dos consultas sobre Food Store: una de resumen

(agregación) y una con subconsulta. La especificación debe fijar explícitamente: tablas involucradas, filtro de

borrado lógico, columnas de salida, orden y, si corresponde, criterio de corte (LIMIT, HAVING).

3. Escribir, de forma independiente, una segunda versión propia (o pedirle a la IA una alternativa con distinta

estructura: subconsulta vs. join, por ejemplo) que responda la misma pregunta.

4. Verificar la equivalencia de resultados entre ambas versiones — por ejemplo, comparando el conteo de filas

y una muestra de valores, o con *EXCEPT* entre ambos resultados

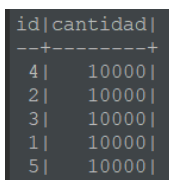
Agregacion:

Genera una query SQL sobre la base foodstore que devuelva ,sin usar select * , el id y cantidad de productos existentes(producto.eliminado=false) que hay en cada categoria, ordenado de mayor a menor cantidad.

Rta ia:

```
SELECT c.id_categoria AS id,  
       COUNT(p.id_producto) AS cantidad  
FROM categoria c LEFT JOIN producto p  
       ON p.id_categoria = c.id_categoria  
       AND p.eliminado = FALSE  
GROUP BY c.id_categoria  
ORDER BY cantidad DESC;
```

salida:

A terminal window showing the output of a SQL query. The output is a table with two columns: 'id' and 'cantidad'. The data is as follows:

id	cantidad
4	10000
2	10000
3	10000
1	10000
5	10000

Alternativa:

```
WITH productos_existentes AS (  
    SELECT id_producto, id_categoria  
    FROM producto  
    WHERE eliminado = FALSE )  
SELECT c.id_categoria AS id,  
       COUNT(pe.id_producto) AS cantidad  
FROM categoria c  
LEFT JOIN productos_existentes pe  
       ON pe.id_categoria = c.id_categoria
```

GROUP BY c.id_categoria

ORDER BY cantidad DESC;

salida:

id	cantidad
4	10000
2	10000
3	10000
1	10000
5	10000

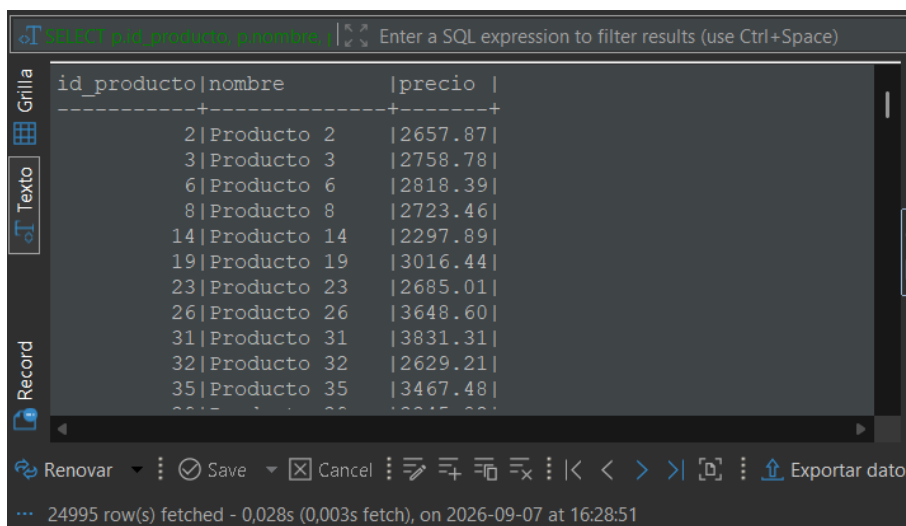
Comparacion (agregación): La query inicial es mas corta (8 lineas) mientras que la alternativa con cte es mas larga (12 lineas) y mas legible, pero ambas son igual de eficientes en cuanto a optimizacion y obtienen la misma salida.

Subconsulta:

Generará una query sql sobre la base foodstore que devuelva ,sin usar select *, id_producto, nombre y precio de los productos existentes(producto.eliminado=false) que tengan un precio mayor al valor promedio de todos los productos.

Rta ia:

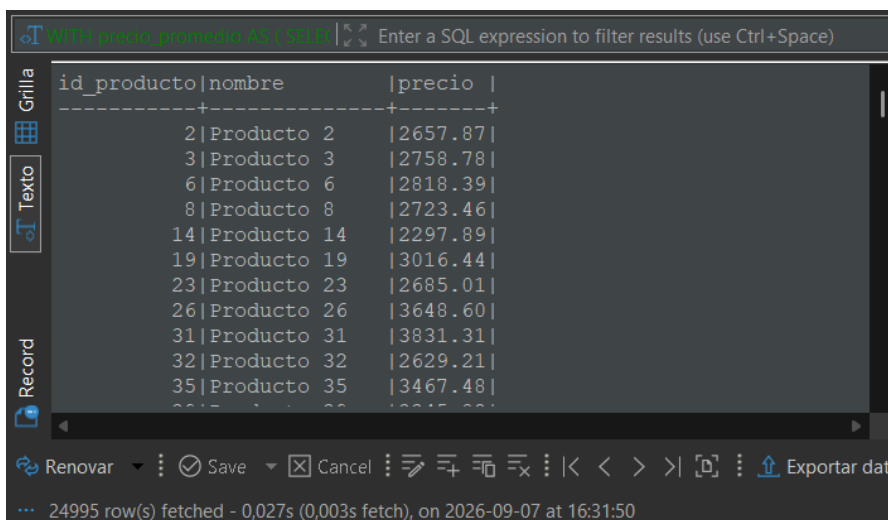
```
SELECT p.id_producto,  
       p.nombre,  
       p.precio  
FROM producto p  
WHERE p.eliminado = FALSE  
      AND p.precio > (SELECT AVG(precio)  
                     FROM producto);
```



id_producto	nombre	precio
2	Producto 2	2657.87
3	Producto 3	2758.78
6	Producto 6	2818.39
8	Producto 8	2723.46
14	Producto 14	2297.89
19	Producto 19	3016.44
23	Producto 23	2685.01
26	Producto 26	3648.60
31	Producto 31	3831.31
32	Producto 32	2629.21
35	Producto 35	3467.48

Alternativa:

```
WITH precio_promedio AS (  
  SELECT AVG(precio) AS promedio  
  FROM producto  
)  
SELECT p.id_producto, p.nombre, p.precio  
FROM producto p  
CROSS JOIN precio_promedio pp  
WHERE p.eliminado = FALSE  
AND p.precio > pp.promedio;
```



id_producto	nombre	precio
2	Producto 2	2657.87
3	Producto 3	2758.78
6	Producto 6	2818.39
8	Producto 8	2723.46
14	Producto 14	2297.89
19	Producto 19	3016.44
23	Producto 23	2685.01
26	Producto 26	3648.60
31	Producto 31	3831.31
32	Producto 32	2629.21
35	Producto 35	3467.48

Comparacion(subconsulta):

La query inicial (7 líneas), mientras que la variante con CTE + CROSS_JOIN es más larga (9 líneas) y más legible, pero ambas son igual de eficientes en cuanto a optimización y obtienen la misma salida. Es importante aclarar que la query alternativa basa su eficiencia en que la tabla del join (el promedio calculado en el CTE) tiene una sola fila, por lo que este modelo recorre muy poco por cada fila externa, el promedio se calcula una única vez y no se recalcula por fila. El costo podría crecer si la tabla interna del join tuviera muchas filas.